

A Decentralized Autonomous Organization Framework for Real Estate Tokenization on Ethereum

Christover Angelo Lasut
Dept. of Electrical Engineering
Faculty of Engineering, Universitas
Indonesia
Kampus Baru UI, Depok 16424
Email: christoverangelo1.com

Matthew Immanuel Sitorus
Dept. of Electrical Engineering
Faculty of Engineering, Universitas
Indonesia
Kampus Baru UI, Depok 16424
Email: noels170rus2005@gmail.com

Syahmi Hamdani
Dept. of Electrical Engineering
Faculty of Engineering, Universitas
Indonesia
Kampus Baru UI, Depok 16424
Email: syahmi.hamdani@ui.ac.id

Abstract—Real-world asset (RWA) tokenization has emerged as a transformative mechanism for fractionalizing traditionally illiquid assets such as real estate into tradeable digital tokens on public blockchain networks. However, existing commercial platforms suffer from a fundamental structural weakness: governance and asset management decisions remain under centralized operator control, reintroducing the counterparty risk and information asymmetry that blockchain technology was designed to eliminate. This paper presents PropDAO, a decentralized application (dApp) built on the Ethereum blockchain that tightly integrates fractional real estate tokenization with a fully on-chain Decentralized Autonomous Organization (DAO) governance system. The platform is underpinned by four modular smart contracts — PropertyToken, PropertyDAO, PropertyRegistry, and DividendDistributor — developed in Solidity v0.8.28 using the Hardhat development framework and OpenZeppelin v5 contract libraries. The DAO contract, based on OpenZeppelin's Governor extensions, enables token-weighted voting on all material asset management decisions, while the dividend distributor automates proportional rental yield distribution trustlessly using a pull-over-push payment pattern. Metadata for physical assets is stored on-chain with IPFS content-addressed document links via the PropertyRegistry contract. The complete system was deployed and validated on the Ethereum Sepolia public testnet. Evaluation results indicate that the platform successfully eliminates dependence on centralized intermediaries within its technical scope and provides a transparent, auditable, gas-measured, and user-accessible infrastructure for fractional real estate investment governed entirely by its community of token holders.

Keywords—Blockchain, Decentralized Autonomous Organization (DAO), Real-World Asset Tokenization, Ethereum, Smart Contracts

I. INTRODUCTION

The global real estate market represents one of the largest and most valuable asset classes in the world, yet it remains fundamentally inaccessible to the majority of investors. High capital requirements for outright property purchase, geographic barriers, illiquid secondary markets, and opaque management structures collectively exclude a large segment of potential participants. Real-world asset (RWA) tokenization has emerged as a promising structural solution: by representing fractional ownership of physical assets as digital

tokens on a public blockchain ledger, it becomes possible to divide a property into thousands of transferable shares that can be purchased, managed, and traded with the transparency and programmability of decentralized infrastructure [1], [2].

The foundational technology enabling this capability is the Ethereum blockchain [3], a programmable distributed ledger platform that introduced the concept of smart contracts — deterministic, self-executing programs stored on-chain whose terms are enforced by the network's consensus mechanism without the need for any trusted third party [4]. The ERC-20 token standard [5] built on Ethereum provides a well-established interface for fungible tokens, making it a natural choice for representing fractionalized ownership shares in a real-world asset.

Despite the conceptual promise of RWA tokenization, existing commercial platforms such as RealT and Landshare retain centralized governance structures. Property management decisions, including maintenance approvals, tenant selection, rent policy, and dividend distribution, are made unilaterally by platform operators rather than by the investors themselves. This centralization reintroduces the very counterparty risk and financial mismanagement that blockchain technology was designed to eliminate, and has led to documented cases of investor disputes in the broader digital asset space [2].

Decentralized Autonomous Organizations (DAOs) offer a well-studied model for collectively governing shared resources through transparent, on-chain voting mechanisms enforced by smart contracts [6], [7]. Token holders in a DAO can submit proposals, vote proportionally to their stake, and have outcomes automatically executed by the network — removing the need for any trusted human administrator. Yet the deliberate application of DAO governance to tokenized real-world assets remains largely unexplored in production systems.

This paper makes the following specific contributions:

- We design and implement a modular four-contract smart contract architecture that combines ERC-20 fractional ownership tokens with a token-weighted DAO governance mechanism and automated dividend distribution, all built on OpenZeppelin's battle-tested contract libraries.

- We introduce a pull-over-push dividend distribution mechanism that distributes rental yield proportionally to token holders in a reentrancy-safe and gas-efficient manner.
- We integrate decentralized asset metadata storage using IPFS [8] via a dedicated on-chain PropertyRegistry contract, providing immutable, auditable links to legal documents and property records.
- We present a fully functional dApp frontend built with React.js and integrated with MetaMask, enabling investors to participate in governance and claim dividends without prior blockchain expertise.
- We deploy and validate the entire system on the Ethereum Sepolia public testnet, providing verified on-chain proof of each contract's deployment and functionality, with gas cost measurements for key operations.

The remainder of this paper is organized as follows: Section II reviews the relevant literature. Section III describes the system architecture and smart contract design. Section IV details the implementation methodology and experimental setup. Section V presents results and evaluation. Section VI concludes and outlines future work.

II. RELATED WORK

A. Blockchain Foundations and Smart Contracts

Nakamoto's seminal Bitcoin whitepaper [9] first demonstrated that trustless peer-to-peer electronic transactions are possible without a central authority through cryptographic proof and distributed consensus. While Bitcoin established the viability of decentralized value transfer, Buterin [3] extended this paradigm to general-purpose programmable computation with the Ethereum platform, introducing smart contracts as Turing-complete programs that run deterministically on a decentralized virtual machine. Antonopoulos and Wood [10] provide a comprehensive reference for Ethereum's technical architecture, covering the Ethereum Virtual Machine (EVM), gas economics, and development patterns relevant to this work. Szabo [4] originally conceptualized smart contracts as digital mechanisms that formalize, verify, and enforce contract terms without trusted intermediaries — a philosophy directly embodied in the PropDAO governance system.

B. RWA Tokenization and Traditional Finance

Aloshyna and Aleksandrov [1] examine how asset tokenization serves as a bridge between conventional financial systems and the emerging blockchain ecosystem. The authors demonstrate that representing physical assets as digital tokens on a distributed ledger significantly lowers the barrier to entry for investors through fractionalization, improves liquidity in traditionally illiquid markets, and enables near-instant settlement with enhanced transparency. However, the paper acknowledges persistent regulatory challenges around investor protection and the legal recognition of tokenized ownership as key obstacles to mainstream adoption — a limitation that is explicitly acknowledged within its problem scope.

C. Limits of Blockchain-Based Governance for Real Assets

Proskurovska and Birch [2] take a critical analytical stance, arguing that while blockchain provides trustless transaction validation, it does not inherently resolve off-chain problems such as asset valuation disputes, legal enforceability of token-based ownership, and cross-jurisdictional regulatory compliance. The authors highlight the fundamental tension between the automation potential of smart contracts and real-world contingencies — such as physical property damage or tenant default — that require human judgment and legal intervention. This work directly motivates PropDAO's explicit scoping of its governance model to on-chain decisions and its honest acknowledgment of off-chain limitations in Section V-C.

D. Decentralized Finance (DeFi) as Infrastructure

Schär [11] provides a foundational academic reference for the broader DeFi ecosystem, proposing a multi-layered framework that encompasses token standards, decentralized exchanges, lending markets, and on-chain asset management protocols. DeFi's composable, open, and transparent characteristics directly inform PropDAO's design philosophy: by treating the governance token as a standard ERC-20 asset and the dividend distributor as a DeFi-style yield contract, the system inherits DeFi's interoperability potential for future integrations with liquidity pools or lending protocols.

E. Cross-Chain Interoperability for RWA

Guo et al. [12] propose xRWA, a cross-chain protocol enabling tokenized real-world assets to be represented and transferred between different blockchain ecosystems (e.g., Ethereum, BNB Chain, Polygon) without compromising the integrity of ownership records. The framework utilizes cross-chain messaging protocols and wrapped token standards to ensure consistency of asset data across chains. While PropDAO targets a single EVM-compatible network in its current form, xRWA serves as a reference architecture for planned future multi-chain extensibility described in Section VI-B.

F. DAO Governance Challenges

Rikken et al. [7] investigate the governance complexities that arise in DAO structures on blockchain networks. The paper identifies fundamental tensions in DAO design: the trade-off between operational efficiency and decentralized decision-making, risks of voter apathy (where quorum is consistently not reached), and plutocracy where wealthy token holders systematically dominate governance outcomes. These findings directly inform PropDAO's governance parameterization — particularly the adoption of proposal thresholds, configurable time-locked voting periods, and proportional voting weights. Chohan [6] provides a complementary analysis of early DAO failures, most notably the 2016 "The DAO" exploit [13], identifying both technical vulnerabilities (reentrancy attacks) and governance design flaws (insufficient checks on proposal execution) that this project actively mitigates through OpenZeppelin's audited Governor implementation and ReentrancyGuard.

G. Smart Contract Security

Atzei, Bartoletti, and Cimoli [13] present a comprehensive taxonomy of attack vectors against Ethereum smart contracts, including reentrancy, integer overflow/underflow, timestamp dependence, and transaction-ordering dependence. This survey directly shaped PropDAO's security design: ReentrancyGuard is applied to the DividendDistributor contract, Solidity v0.8.x's built-in overflow protection is leveraged throughout, and OpenZeppelin's access control primitives (Ownable, onlyOwner) guard all privileged functions. Wöhrer and Zdun [14] further provide concrete security-oriented design patterns for Solidity contracts that informed the pull-payment architecture adopted in the dividend distributor. Weyl [15] provides the theoretical basis for Quadratic Voting as a future governance enhancement to counteract plutocracy.

H. Decentralized Storage

Benet [8] introduces InterPlanetary File System (IPFS), a content-addressed, versioned, peer-to-peer file system that assigns each piece of content a deterministic hash-based identifier. PropDAO uses IPFS to store legal documents, property photographs, and valuation reports associated with registered real estate assets. The PropertyRegistry contract stores the IPFS content hash (CID) on-chain, ensuring that any reference to a document is immutable and tamper-evident, while the actual file data is served through a distributed network (via Pinata gateway) rather than stored expensively on-chain.

I. Gap Addressed by This Work

Prior literature tends to discuss asset tokenization and DAO governance as entirely separate domains. Schär [11] and Alosyna and Aleksandrov [1] focus on tokenization economics; Rikken et al. [7] and Chohan [6] focus on DAO governance theory. No existing platform combines both paradigms into a single cohesive production system for real estate management. The novelty of PropDAO lies in this integration: investors simultaneously hold fractionalized ownership of physical assets and exercise direct, on-chain governance rights over the management of those assets — eliminating trusted intermediaries and substantially reducing financial mismanagement risk.

III. SYSTEM ARCHITECTURE

A. System Overview

PropDAO is architected as a three-layer system: (1) a blockchain layer consisting of four modular smart contracts deployed on Ethereum, (2) a middleware layer providing Web3 connectivity via Ethers.js RPC calls, and (3) a presentation layer implemented as a React.js single-page application (SPA) authenticated via MetaMask.

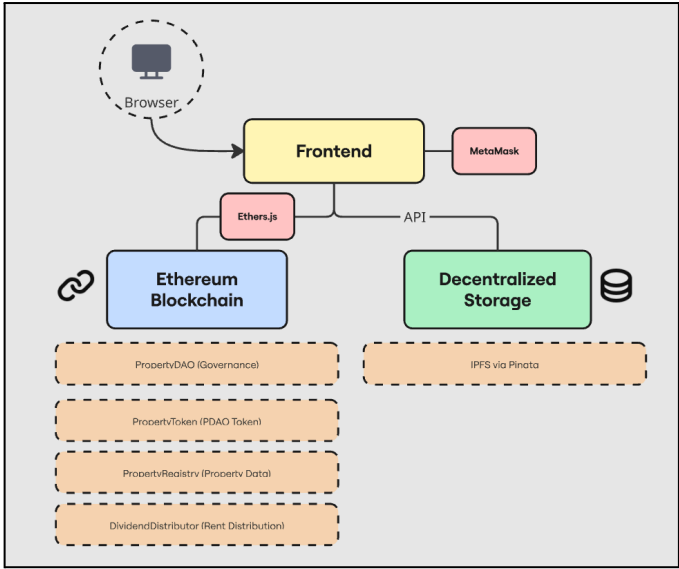


Fig. 1. System Architecture Diagram

B. Smart Contract Design

The system is designed using four modular smart contracts that interact through well-defined interfaces. This modular approach enforces separation of concerns and allows individual components to be upgraded or replaced without affecting the rest of the system.

TABLE I. Smart Contract Responsibilities

Contract	Solidity Basis	Primary Responsibilities
PropertyToken.sol	ERC-20, ERC-20Votes (OZ v5)	Minting fractional ownership tokens; managing transfers; recording on-chain voting power snapshots
PropertyDAO.sol	OZ Governor + GovernorVotes + GovernorVotesQuorumFraction	Accepting governance proposals; managing token-weighted voting; auto-executing approved outcomes
DividendDistributor.sol	Custom ReentrancyGuard (OZ)	Receiving ETH rental income; proportionally distributing yield via pull-payment pattern
PropertyRegistry.sol	Custom + Ownable (OZ)	Registering asset metadata: location, valuation, and IPFS CID for legal documents

C. Governance Mechanism

The governance module is implemented using OpenZeppelin's Governor contract family with GovernorVotes and GovernorVotesQuorumFraction extensions. Voting power

is derived from PropertyToken balances at a snapshot block preceding each proposal's creation, preventing flash-loan manipulation of voting weight [16]. Proposals pass if they simultaneously meet the quorum threshold (minimum percentage of total token supply that must vote) and achieve a simple majority of "For" votes within a configurable voting period. Upon a successful vote and a mandatory time-lock delay, execution is triggered on-chain without any manual intervention.

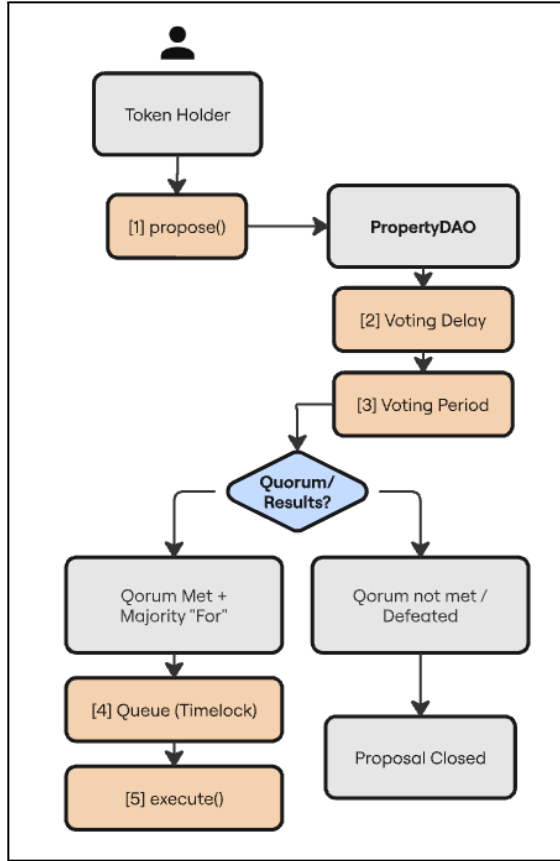


Fig. II. DAO Proposal Lifecycle

D. Dividend Distribution Mechanism

The DividendDistributor contract employs a pull-over-push payment pattern [10], [14]. When the contract receives ETH via `payRent()`, it does not immediately push funds to investors' wallets (which would be gas-prohibitive and vulnerable to reentrancy). Instead, it updates an internal per-share accumulator, and investors call `claimDividend()` at their own convenience to withdraw their proportional accumulated share. The contract is protected by OpenZeppelin's ReentrancyGuard to prevent re-entrancy attacks on the withdrawal function.

E. Decentralized Metadata storage

Property metadata — physical address, valuation, and legal document bundles — is stored off-chain on IPFS [8] and pinned via the Pinata service for persistence. Only the resulting IPFS Content Identifier (CID) is stored on-chain in

the PropertyRegistry contract. This approach balances cost (storing large binary files on-chain is prohibitively expensive at Ethereum gas prices) with immutability (the CID cryptographically commits to the exact content of the off-chain document; any tampering produces a different CID).

IV. IMPLEMENTATION AND EXPERIMENTAL SETUP

A. Development Methodology

This project adopted an Agile development methodology using a four-sprint iteration approach [17]. This was chosen due to the multi-layer nature of the project — integrating blockchain infrastructure, smart contracts, and a user-facing interface — making incremental testing and rapid adjustment essential.

B. Risk Management

To ensure the stability and security of the proposed decentralized system, a comprehensive evaluation of potential operational and technical threats was conducted. The identified vulnerabilities span smart contract vulnerabilities, user engagement challenges, development timeline dependencies, and external compliance shifts. To proactively address these concerns, strategic countermeasures have been integrated directly into the system's architecture and development lifecycle. A detailed breakdown of these critical factors, including their estimated probability, potential severity, and corresponding preventative measures, is systematically cataloged in

TABLE II. Risk Identification and Mitigation

Risk	Likelihood	Impact	Mitigation Strategy
Smart contract bug or exploit	Medium	High	Comprehensive unit testing; OpenZeppelin audited libraries; manual code review
Voter apathy (quorum not reached)	High	Medium	Configurable quorum threshold; token delegation reminder in UI
Frontend-blockchain integration delays	Medium	Medium	Mock ABI during early sprints; parallel iterative development
Regulatory changes to digital assets	Low	High	Modular contract design; governance logic separable from token contract

C. Technology Stack

To build a secure, transparent, and decentralized ecosystem, modern and industry-standard Web3 technology stack are used. The platform's backend relies on Ethereum's EVM network and audited Solidity smart contracts to handle all transactions and data securely. The user interface is built with React.js, allowing seamless wallet connection via MetaMask and smooth interaction with the blockchain through Ethers.js. To guarantee true data ownership and eliminate central points of failure, all property deeds and legal documents are stored permanently using decentralized IPFS storage.

TABLE III. Technology Stack

Layer	Technology
Blockchain Network	Ethereum (EVM) — Sepolia Testnet
Smart Contract Language	Solidity
Development Framework	Hardhat
Contract Libraries	OpenZeppelin Contracts
Frontend Framework	React.js (Vite bundler)
Web3 Integration	Ethers.js
Wallet Authentication	MetaMask browser extension
Decentralized Storage	IPFS via Pinata Gateway
Package Manager	Node.js / npm

D. Smart Contract Implementation Details

PropertyToken.sol is implemented as a standard ERC-20 token extended with ERC20Votes and ERC20Permit from OpenZeppelin v5. The ERC20Votes extension adds checkpoint-based voting power snapshots, enabling the PropertyDAO to query a holder's balance at any past block without being susceptible to flash-loan attacks [16]. The initial supply is minted to the deployer address at construction time. Token holders must call `delegate()` (delegating to themselves or another address) to activate their voting power — a design requirement of OpenZeppelin's Governor that ensures explicit opt-in to governance participation.

PropertyDAO.sol is constructed by composing three OpenZeppelin extensions:

- Governor: Core proposal lifecycle management (propose, vote, execute)

- GovernorVotes: Reads voting power from PropertyToken's ERC20Votes checkpoints
- GovernorVotesQuorumFraction: Enforces a percentage-based quorum of the total token supply

DividendDistributor.sol implements ReentrancyGuard to prevent recursive withdrawal exploits [13]. The contract maintains per-address debtBalances and a global dividendPerToken accumulator. When `payRent()` is called with ETH, the contract increments dividendPerToken by the per-token share. When `claimDividend()` is called, the contract computes and transfers the investor's accrued yield, then resets their debtBalance to the current accumulator value. The contract reverts if called with zero ETH in `payRent()`.

PropertyRegistry.sol extends OpenZeppelin's Ownable contract. Only the owner (typically the deployer/admin) can register or delete properties. Each property entry stores a location string, valuation string, and an IPFS hash string. Properties are assigned sequential integer IDs. The contract emits events on registration, update, and deletion, providing a complete on-chain audit trail queryable by the frontend.

E. Frontend Implementation

The user interface is a single-page application (SPA) built with React.js and bundled with Vite. It is structured into three primary views:

- Investor Dashboard: Displays the connected wallet address, RWA token balance, fractional ownership percentage, and claimable dividend amount — all fetched in real time from the blockchain via Ethers.js RPC calls. Users can claim their accumulated dividends directly from this view.
- Governance Page: Allows users to browse active, pending, defeated, and executed proposals. Token holders meeting the minimum threshold can submit new proposals via a structured form. Voting progress is visualized with For/Against/Abstain vote totals and percentage bars.
- Admin Panel (deployer-only): Provides a restricted interface for registering new properties, including an IPFS upload pipeline that pins documents and images to Pinata before writing the resulting CID to the PropertyRegistry contract.

F. Web3 Integration Pattern

Authentication uses MetaMask as the primary wallet provider via the EIP-1193 provider API. Connecting the wallet retrieves the user's public address as the identity, without any username, password, or centralized session management. Write operations (casting a vote, claiming dividends, submitting a proposal, registering a property) trigger MetaMask signature requests, ensuring explicit user consent for every state-mutating action. Read operations (checking balances, fetching proposal data, reading registry entries) are executed via a public Alchemy RPC endpoint without gas fees.

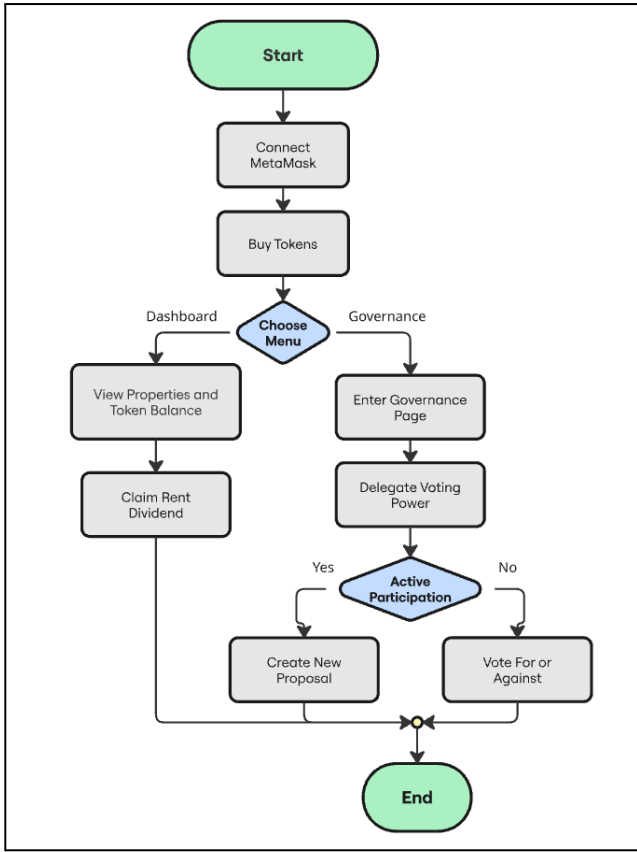


Fig. III. User Interaction Flowchart

G. Unit Testing

Unit Testing was conducted within the Hardhat test environment, with each contract tested in isolation against a fresh local blockchain snapshot:

- Verified initial supply, transfer restrictions, and `delegate()` → `getVotes()` checkpoint behavior.
- Verified that proposals with insufficient quorum are defeated; proposals with majority "For" votes are queued and executed; proposals in the wrong state correctly revert.
- Verified proportional dividend calculation across multiple token holders; reentrancy attack simulation confirms `ReentrancyGuard` protection; `zero-ETH payRent()` correctly reverts.
- Verified sequential property ID assignment, event emission on registration/deletion, and access control preventing non-owner registration.

Integration Testing validated the full cross-contract interaction chain. Proposal targeting `payRent()` is correctly executable after passing the voting period, and the registry's state is correctly read by the frontend after a new property registration.

V. RESULTS AND DISCUSSION

A. Feature Evaluation Against Objectives

All six primary project objectives stated in Section I were confirmed through on-chain interaction and integration testing on the Sepolia testnet.

TABLE IV. Feature Evaluation

Objective	Outcome	Evidence
Fractional ownership tokenization (ERC-20)	Met	Token balances and transfers confirmed on Blockscout
DAO governance with token-weighted voting	Met	Full proposal lifecycle tested (propose → vote → execute)
Automated dividend distribution	Met	<code>claimDividend()</code> correctly distributes proportional ETH yield
Property metadata with IPFS document links	Met	Properties registered with IPFS CID; readable from frontend
MetaMask wallet-only authentication	Met	No credentials required; wallet address used as identity
Sepolia public testnet deployment	Met	All four contracts deployed with verified transaction hashes

B. Gas Cost Measurement

Gas consumption was measured for key operations on the Sepolia testnet. These measurements are critical for understanding the cost of using the platform on Ethereum mainnet and motivating the Layer-2 scaling strategy outlined in future work. At mainnet gas prices (e.g., 20 gwei), a single `castVote()` transaction would cost approximately 4.50–9.00 USD, representing a meaningful barrier for small token holders. This directly motivates the Layer-2 deployment strategy discussed in Section VI.

C. Competitive Advantage Evaluation

While current fractional real estate platforms offer digital property ownership, they often rely on centralized management and hidden, closed-source code. This is eliminated by introducing full on-chain DAO governance and automated, trustless payout systems. Unlike its competitors, it ensures complete transparency by making its smart contracts open-source and publicly hosting all property documentation on IPFS. This setup gives users full control over their investments without relying on a central authority.

TABLE V. Feature Comparison with Existing Platforms

Feature	RealT (existing)	Landshare (existing)	PropDAO
Fractional real estate ownership	Yes	Yes	Yes
On-chain DAO (all governance decisions)	Centralized	Limited	Full on-chain
Automated trustless dividend distribution	Partial	Partial	Trustless pull-payment
Open-source auditable smart contracts	No	No	Yes
IPFS-linked property document storage	No	No	Yes
Wallet-only (no custodial) authentication	Yes	Yes	Yes
Multi-chain support	No	BNB only	Planned
Layer-2 / low gas fee support	No	No	Planned

D. Security Analysis

Reentrancy Protection: OpenZeppelin's ReentrancyGuard modifier is applied to `claimDividend()`. The function follows the Checks-Effects-Interactions pattern [13]: the investor's debt balance is updated before the ETH transfer is executed, preventing recursive re-entry.

Integer Overflow/Underflow: Mitigated automatically by Solidity v0.8.x's built-in arithmetic overflow checks, which revert the transaction on overflow/underflow without requiring explicit SafeMath library usage [18].

Access Control: The Ownable modifier on PropertyRegistry and deployer-only functions ensures that critical asset management operations cannot be called by arbitrary external actors. The PropertyDAO functions as the sole intended future owner of privileged operations.

Flash-Loan Attack Prevention: The GovernorVotes extension reads voting power from a token balance snapshot taken at the block prior to proposal creation, making it impossible to acquire tokens in the same block as a proposal to manipulate voting outcomes [16].

Front-Running Mitigation: The DAO's voting period and time-lock delay provide a window during which the community can observe and react to any suspicious proposal before it is executed.

E. Limitations and Challenges

1. **Layer-1 Gas Costs:** On Ethereum mainnet at moderate gas prices, governance participation (casting a vote, claiming dividends) can become cost-prohibitive for small token

holders, potentially exacerbating plutocracy rather than mitigating it. This is a fundamental constraint of L1 Ethereum rather than a design flaw.

2. **Off-Chain Legal Enforceability:** As explicitly scoped in Section I, the platform lacks a legally binding bridge between digital tokens and the physical property deed. Smart contracts cannot autonomously enforce real-world contingencies such as evicting a defaulting tenant or responding to physical property damage.

3. **Voter Apathy and Quorum Risk:** During integration testing, quorum was only reliably achieved when the deployer account (holding a large initial token allocation) participated in governance. A highly fragmented token distribution in production would require active community engagement mechanisms beyond the current scope. Rikken et al. [7] identify this as a systemic challenge across all DAO implementations.

4. **User Accessibility:** Relying entirely on MetaMask as the authentication and signing mechanism excludes non-crypto-native investors. Mandatory gas fee management and private key custody present significant UX barriers for mainstream real estate investors accustomed to traditional financial interfaces [19].

5. **Oracle Dependency for Real-World Data:** Property valuations stored in PropertyRegistry are entered manually at registration time and are not dynamically updated. Integrating a decentralized oracle network [20] would be required to automate valuation updates, enabling use cases such as automated loan-to-value ratio checks or index-linked governance thresholds.

VI. CONCLUSION AND FUTURE WORK

A. Conclusion

This paper presented PropDAO, a decentralized real estate tokenization platform governed entirely by an on-chain DAO mechanism built on the Ethereum blockchain. The system demonstrates that a four-contract modular architecture (PropertyToken (ERC-20 with governance vote tracking), PropertyDAO (OpenZeppelin Governor with quorum-based voting), DividendDistributor (automated pull-payment rental yield), and PropertyRegistry (IPFS-linked asset metadata)) can collectively eliminate dependence on centralized intermediaries that characterizes existing RWA tokenization platforms.

The platform was successfully deployed to the Ethereum Sepolia testnet and validated through both unit and integration testing. Feature evaluation confirmed that all primary project objectives were achieved. Gas cost measurements provide empirical data for informed future deployment decisions. Competitive analysis demonstrated a clear differentiation from existing market platforms, particularly in the completeness and enforceability of on-chain governance, and in the trustlessness of automated dividend distribution.

The fundamental insight of this work is that combining DAO governance with RWA tokenization into a single cohesive system is not merely theoretically sound but practically implementable using current EVM tooling and

open-source contract libraries. This prototype provides a technically validated foundation for future development toward a market-ready platform.

REFERENCES

- [1] A. Aloshyina and A. Aleksandrov, "Tokenization of real-world assets (RWA): A bridge between traditional finance and blockchain," *Baltic Journal of Legal and Social Sciences*, vol. 2025, no. 1 Special issue, pp. 41–49, 2025, doi: 10.30525/2592-8813-2025-spec-4.
- [2] A. Proskurovska and K. Birch, "Tokenization of everything? Exploring the limits of blockchain technologies in the governance of financial markets and assets," *Finance and Society*, pp. 1–21, 2025, doi: 10.1017/fas.2025.10026.
- [3] V. Buterin, "A next-generation smart contract and decentralized application platform," Ethereum.org, 2014. [Online]. Available: <https://ethereum.org/en/whitepaper/>
- [4] N. Szabo, "Formalizing and securing relationships on public networks," *First Monday*, vol. 2, no. 9, Sep. 1997, doi: 10.5210/fm.v2i9.548.
- [5] F. Vogelsteller and V. Buterin, "EIP-20: Token Standard," *Ethereum Improvement Proposals*, Nov. 2015. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-20>
- [6] U. W. Chohan, "The decentralized autonomous organization and governance issues," *Discussion Paper Series: Notes on the 21st Century*, SSRN, Dec. 2017, doi: 10.2139/ssrn.3082055.
- [7] O. Rikken, M. Janssen, and Z. Kwee, "Governance challenges of blockchain and decentralized autonomous organizations," *Digital Policy, Regulation and Governance*, vol. 21, no. 4, pp. 397–417, 2019, doi: 10.3233/IP-190154.
- [8] J. Benet, "IPFS - Content addressed, versioned, P2P file system," *arXiv preprint arXiv:1407.3561*, 2014. [Online]. Available: <https://arxiv.org/abs/1407.3561>
- [9] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [10] A. M. Antonopoulos and G. Wood, *Mastering Ethereum: Building Smart Contracts and DApps*. Sebastopol, CA: O'Reilly Media, 2018.
- [11] F. Schär, "Decentralized finance: On blockchain- and smart contract-based financial markets," *Federal Reserve Bank of St. Louis Review*, vol. 103, no. 2, pp. 153–174, Second Quarter 2021, doi: 10.20955/r.103.153-74.
- [12] Y. Guo, H. Zhu, M. Xu, X. Cheng, and B. Xiao, "xRWA: A cross-chain framework for interoperability of real-world assets," *arXiv preprint arXiv:2509.12957v3*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.12957>
- [13] N. Atzei, M. Bartoletti, and T. Cimoli, "A survey of attacks on Ethereum smart contracts (SoK)," in *Proc. 6th Int. Conf. Principles of Security and Trust (POST), Lecture Notes in Computer Science*, vol. 10204, Springer, Berlin, Heidelberg, 2017, pp. 164–186, doi: 10.1007/978-3-662-54455-6_8.
- [14] M. Wöhler and U. Zdun, "Smart contracts: Security patterns in the Ethereum ecosystem and Solidity," in *Proc. 2018 IEEE 1st Int. Workshop on Blockchain Oriented Software Engineering (IWBOSE)*, 2018, pp. 2–8, doi: 10.1109/IWBOSE.2018.8327565.
- [15] E. G. Weyl, "The robustness of quadratic voting," *Public Choice*, vol. 172, no. 1–2, pp. 75–107, Feb. 2017, doi: 10.1007/s11127-017-0405-4.
- [16] OpenZeppelin, "ERC20Votes," *OpenZeppelin Contracts Documentation*, 2024. [Online]. Available: <https://docs.openzeppelin.com/contracts/5.x/api/token/erc20#ERC20Votes>
- [17] K. Beck et al., "Manifesto for agile software development," 2001. [Online]. Available: <https://agilemanifesto.org/>
- [18] Ethereum Foundation, "Solidity v0.8.0 breaking changes — checked arithmetic," *Solidity Documentation*, 2020. [Online]. Available: <https://docs.soliditylang.org/en/v0.8.0/080-breaking-changes.html>
- [19] A. Murray, D. Kim, and J. Combs, "The promise of a decentralized internet: What is Web3 and how can firms prepare?" *Business Horizons*, vol. 66, no. 2, pp. 191–202, Mar.–Apr. 2023, doi: 10.1016/j.bushor.2022.06.002.
- [20] S. Ellis, A. Juels, and S. Nazarov, "ChainLink: A decentralized oracle network," *Chainlink Whitepaper*, 2017. [Online]. Available: <https://chain.link/whitepaper>